

GDB调试工具使用体验

好，代码编译通过了，镜像也生成了，QEMU 也能跑起来了。但是，如果代码运行的结果不符合预期，或者干脆就崩溃了，我们该怎么办？在应用程序开发中，我们可能会用 `printf` 打印日志，或者用 IDE 提供的图形化调试器。但在操作系统的底层世界里，我们最得力的伙伴是命令行调试工具——GDB。

GDB工具的安装

首先，我们得确保手头有合适的工具。我们需要一个能理解 RISC-V 架构指令集的 GDB。幸运的是，在我们之前下载的预编译工具链里，它已经准备好了。如果你不确定它在哪里，可以敲入以下命令来寻找：

```
sudo find / -name "riscv64-unknown-elf-gdb" 2>/dev/null
```

（你会发现它安静地待在工具链目录的 `bin` 文件夹里）

前置准备

如何窥探一个正在运行的内核？

想象一下，你要调试一个普通的程序，比如一个 C++ 小程序。你可能是直接启动调试器来运行它。但我们现在要调试的是一个操作系统内核，它本身就是运行在硬件之上的“环境”。我们怎么调试一个“环境”呢？

这里的一个巧妙思路是“远程调试”。我们不直接调试硬件上的内核，而是让 QEMU 这个模拟器来帮忙。QEMU 可以扮演一个“被调试的目标”，它按照我们的要求启动内核，然后在某个端口上等待；同时，我们启动 GDB 这个“调试器”，去连接 QEMU 等待的那个端口。这样一来，GDB 就能向我们报告 QEMU 内部那个虚拟 CPU 的一举一动，让我们像调试普通程序一样调试内核。

这就意味着，我们需要两个终端窗口：一个用来运行 QEMU（作为被调试目标），另一个用来运行 GDB（作为调试器）

同时与两个程序打交道

怎么方便地同时看两个终端窗口呢？如果你使用的是 VS Code 和 Ubuntu 虚拟机，可以直接开启两个终端，你也可以选择一些工具在同一个窗口里划分出两个窗格进行使用，这里推荐一个终端利器：`tmux`（如果没有使用 `apt install tmux` 下载）。

在命令行中输入 `tmux` ，你就进入了一个终端复用会话。这个界面本身看起来和普通终端没什么不同，但它的强大之处在于可以分割屏幕。按下 `Ctrl+B` ，然后再按 `%` ，你就会发现屏幕被垂直切成了两半。现在我们就拥有了两个命令行窗格，按 `Ctrl+B` 再按方向键，你就可以在两个窗格之间切换焦点。

`tmux` 的大部分操作都依赖于一个前缀键（默认是 `ctrl+B` ），按下前缀键之后，我们可以按下其他的功能键来完成操作，以下列举一些常用的操作：

tmux 快捷键与会话管理速查表

操作类别	快捷键/命令	说明
分割窗格	<code>Ctrl+B %</code>	垂直分割为左右两个窗格
	<code>Ctrl+B "</code>	水平分割为上下两个窗格
切换焦点	<code>Ctrl+B</code> + 方向键 (<code>↑↓←→</code>)	在窗格间切换
	<code>Ctrl+B ;</code>	切换到上一个活动窗格
调整布局	<code>Ctrl+B { / Ctrl+B }</code>	与前/后一个窗格交换位置
	<code>Ctrl+B Ctrl+</code> 方向键	调整窗格大小（持续按住）
关闭/管理窗格	<code>Ctrl+D</code> 或输入 <code>exit</code>	关闭当前窗格
	<code>Ctrl+B x</code>	强制关闭当前窗格（需确认）
	<code>Ctrl+B z</code>	最大化当前窗格，再次按下恢复
会话管理	<code>Ctrl+B d</code>	脱离当前会话，任务后台运行
	<code>tmux attach -t 0</code>	重新连接到编号为 0 的会话
	<code>tmux new -s session_name</code>	创建新会话（指定名称）
	<code>tmux ls</code>	列出所有后台会话
	<code>tmux attach -t session_name</code>	连接到指定名称的会话
历史输出查看	<code>Ctrl+B [</code>	进入复制模式，滚动历史输出
	方向键 / <code>PageUp</code> / <code>PageDown</code>	滚动查看历史输出
	<code>q</code>	退出复制模式

`tmux` 的强大之处在于其灵活的会话与窗格管理能力，建议多尝试上述快捷键，提升终端操作效率。

开调！

左边窗格，我们输入 `make debug` 。这个命令会启动 QEMU ，但特别注意，这里的参数 `-s` 会让虚拟 CPU 一启动就立刻暂停，乖乖地等我们发号施令；而参数 `-s` 则告诉 QEMU ：“打开 1234 端口，准备接受 GDB 的连接”。

右边窗格，我们输入 `make gdb` 。这个命令其实是一系列操作的集合：

`file bin/kernel` ：让 GDB 加载我们编译好的内核文件，这个文件里包含宝贵的调试符号（函数名、变量名等）。

`set arch riscv:rv64` ：告诉 GDB ，我们要调试的是 RISC-V 64 位的程序。

`target remote localhost:1234` ：让 GDB 去连接本机（localhost）的 1234 端口，也就是 QEMU 正在等待我们的地方。

```
@DESKTOP-35HSFEH:lab0$ make debug
```

```
@DESKTOP-35HSFEH:lab0$ make
riscv64-unknown-elf-gdb \
  -ex 'file bin/kernel' \
  -ex 'set arch riscv:rv64' \
  -ex 'target remote localhost:1234'
GNU gdb (SiFive GDB-Metal 12.1)
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
This is free software: you can redistribute it and/or modify it under the
terms of the GNU General Public License as published by the Free Software
Foundation.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration.
For bug reporting instructions, please see:
<https://github.com/sifive/freedom/blob/master/CONTRIBUTING.md>
Find the GDB manual and other documentation at:
<http://www.gnu.org/software/gdb/documentation/>
For help, type "help".
Type "apropos word" to search for words related to word.
Reading symbols from bin/kernel...
The target architecture is riscv:rv64.
Remote debugging using localhost:1234
0x0000000000000100 in ?? ()
(gdb)
```

当 GDB 成功连上 QEMU 后，它会告诉我们：“现在程序停在了地址 0x1000 这个地方”。这里其实是 QEMU 内置的固件（BIOS）代码，还没执行到我们的内核。

那么我们的内核代码从哪里开始执行呢？还记得链接脚本吗？它指定了内核的入口地址。我们可以直接在这个地址上打断点，但更简单的方法是使用函数名。因为编译器帮我们把函数名和地址对应了起来（调试符号），所以我们可以直接对 `kern_entry` 函数下断点（`b` 是 `break` 的缩写）：

```
(gdb) b* kern_entry
```

随后执行 `continue` (缩写为 `c`) 开始执行程序，内核会在运行到我们设置好的断点处停止

```
(gdb) c
Continuing.
Breakpoint 1, kern_entry () at kern/init/entry.S:7
7          la sp, bootstacktop
```

此时内核暂停在入口函数的第一条汇编指令处，我们可以检查寄存器状态或反汇编附近的代码。寄存器是 CPU 内部的高速存储单元，存放当前的计算状态和数据流向，对于理解执行上下文极为重要。特别需要注意的是：

`PC`（Program Counter）：指向当前正在执行的指令；

`SP`（Stack Pointer）：表示当前栈顶位置，关乎函数调用和局部变量的存储；

`a0 - a7`：用于传递函数参数的寄存器，可反映初始化阶段的参数传递情况。

我们可以使用 `info registers`（可简写为 `ir`）可查看所有寄存器的值，也可以在 `ir` 后面加上想要查看的寄存器，例如 `ir ra sp`。

```
(gdb) i r # info registers
ra                0x80000a02      0x80000a02
sp                0x8001bd80      0x8001bd80
gp                0x0            0x0
tp                0x8001be00      0x8001be00
t0                0x80200000      2149580800
t1                0x1            1
t2                0x1            1
fp                0x8001bd90      0x8001bd90
s1                0x8001be00      2147597824
a0                0x0            0
a1                0x82200000      2183135232
a2                0x80200000      2149580800
a3                0x1            1
a4                0x800          2048
a5                0x1            1
a6                0x82200000      2183135232
a7                0x80200000      2149580800
s2                0x800095c0      2147521984
s3                0x0            0
s4                0x0            0
s5                0x0            0
s6                0x0            0
s7                0x8            8
s8                0x2000          8192
s9                0x0            0
s10               0x0            0
s11               0x0            0
t3                0x0            0
< quit, c to continue without paging--
t4                0x0            0
t5                0x0            0
t6                0x82200000      2183135232
pc                0x80200000      0x80200000 <kern_entry>
dscratch          Could not fetch register "dscratch"; remote failure reply 'E14'
mucounteren       Could not fetch register "mucounteren"; remote failure reply 'E14'
```

除了上述基本命令之外，GDB 还提供了丰富的调试功能，例如：

- `si` (Step Instruction)：以单条汇编指令为步长单步执行，适用于深入理解底层行为；
- `bt` (BackTrace)：打印函数调用栈，帮助理解执行路径；

- `frame <n>` : 切换到调用栈的第 `n` 层, 查看该层的局部变量和上下文;
- `x/<format> <address>` : 查看内存内容, 如 `x/10x $sp` 表示以十六进制显示栈指针后的10个字。

这些工具在后续复杂的内核调试（如中断处理、虚拟内存、多任务切换等）中尤为重要。

实操建议

在 `kern_entry` 处停下后, 不妨输入几次 `si`, 亲眼看看汇编指令是如何一条条执行的。然后输入 `x/10x $sp`, 看看栈初始化前的内存里到底是什么（很可能是一些垃圾数据）。这种亲手触摸底层的感觉, 是学习操作系统最棒的体验之一。

结语

GDB 是内核调试的利器, 本章仅介绍了基本使用流程。实际上, GDB 生态系统相当丰富: 除了架构专用版本, 还有 `gdb-multiarch` 这样的多架构支持工具; 除了命令行界面, 也有基于 GDB 的图形化前端（如 VS Code 的 GDB 扩展）。在更复杂的调试场景中, 例如需要同时调试内核与用户进程, 开发者还需运用 GDB 的复杂会话管理功能。

内核调试犹如侦探工作, 需要细心观察和逻辑推理。GDB 提供了窥探系统内部状态的能力, 帮助开发者理解代码执行流程、分析问题根源。虽然命令行界面初期可能有些挑战, 但一旦掌握, 这种精准的控制和深入的洞察能力将变得无可替代。

预祝各位在后续开发中能够有效运用调试工具, 既能享受解决复杂问题的成就感, 也能快速定位并修复那些令人困扰的 `bug`。记住, 优秀的开发者不是不写 `bug`, 而是能够快速发现并修复 `bug`。